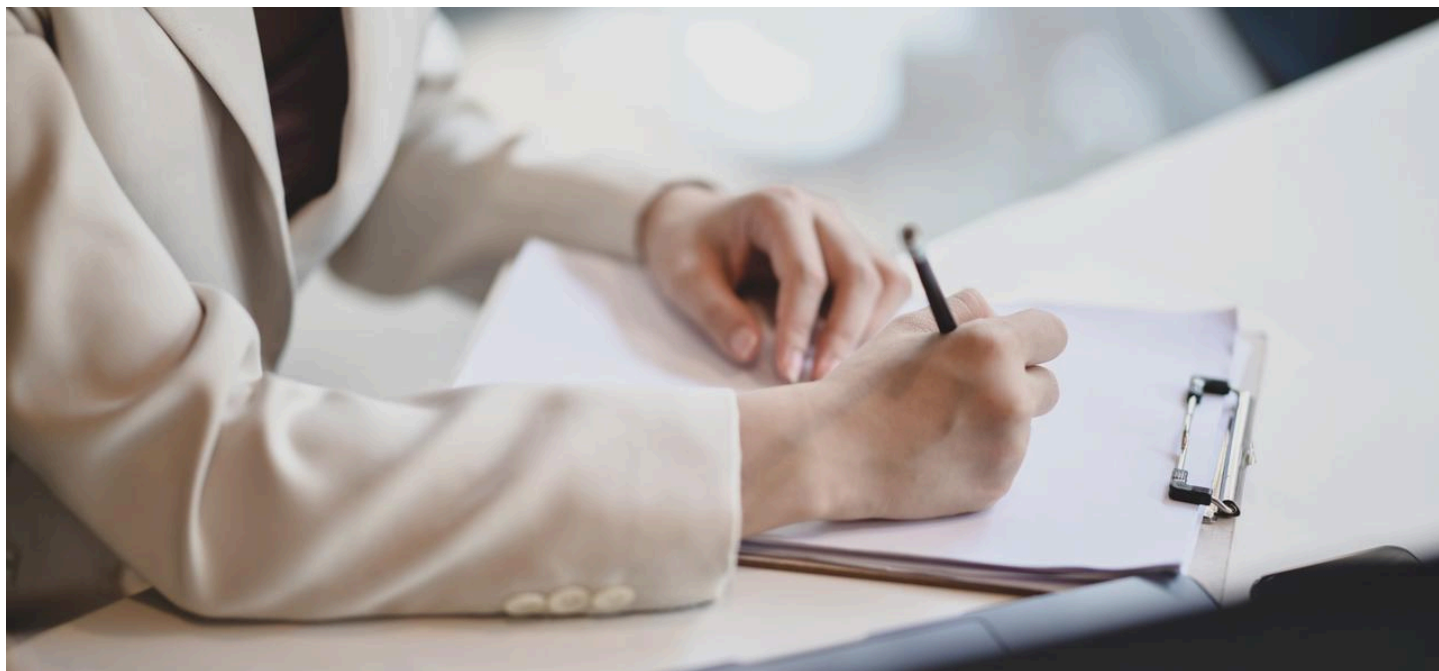




Claude Code 核心概念总结

Claude Code 核心概念总结

来源：Claude Code 官方文档 (code.claude.com/docs)

整理日期：2026-05-16

一、工作原理

智能体循环 (Agentic Loop)

Claude Code 是一个运行在终端的智能编程助手。接收任务后，它通过三个循环阶段工作：

1. **收集上下文 (Gather Context)**：搜索文件、理解代码结构
2. **执行操作 (Take Action)**：编辑文件、运行命令
3. **验证结果 (Verify Results)**：运行测试、确认更改

这三个阶段相互交织，Claude 可将多个工具调用串联成链，并在过程中随时纠偏。**用户可以随时打断并调整方向。**

模型与工具

- **模型**：Sonnet 适合大多数编程任务，Opus 适合复杂架构决策，可通过 `/model` 切换
- **内置工具**： [目 内置工具总览](#)

Claude 可访问的内容

在某目录运行 `claude` 后，Claude 可访问：

- **项目文件**：当前目录及子目录中的文件
- **终端**：所有可运行的命令行工具
- **Git 状态**：当前分支、未提交更改、近期提交历史
- **CLAUDE.md**：项目级的持久化指令文件
- **自动记忆 (Auto Memory)**：Claude 自动保存的学习内容
- **扩展配置**：MCP 服务器、Skills、Subagents、Chrome 插件

高效使用技巧

- **像对话一样工作**：不需要完美的提示词，直接说你想要什么，不对就迭代修正，随时打断重新引导。
 - **前期说清楚**：初始提示越精确，后续修正越少。要引用具体文件、说明约束条件、给出示例模式。
 - **给 Claude 可验证的目标**：提供测试用例、截图或预期输出，Claude 能自我验证时效果最好。
 - **委托而非指挥**：给出背景和方向，信任 Claude 自己决定细节——你不需要指定读哪个文件或运行什么命令。
 - **复杂任务先探索再实现**：用 Plan Mode (`Shift+Tab` 两次切换) 先让 Claude 分析代码库、制定计划，审阅并通过后再执行，效果比直接让它写代码好得多。
-

二、扩展机制

Claude Code 的内置工具覆盖了大多数编码任务。在 Agentic Loop 之上还有一个扩展层，可以使用 [skills](#) 扩展 Claude 知道的内容、使用 [MCP](#) 连接到外部服务、使用 [hooks](#) 自动化工作流，以及将任务卸载给 [subagents](#)。

扩展机制概览

扩展	作用	使用场景	示例
CLAUDE.md	每次会话自动加载的持久上下文	项目规范、"永远做 X"的规则	"使用 pnpm，不用 npm"
Skills	可复用的知识库与可调用的 workflow	可复用内容、参考文档、重复任务	/deploy 运行部署检查
Subagents	在隔离上下文中运行、只返回摘要的独立工作单元	某个子任务产生大量无用输出	读取大量文件但只返回结论
Agent Teams	协调多个独立 Claude Code 会话并行工作	并行研究、新功能开发	同时启动安全、性能、兼容性检查
MCP	连接外部服务和工具的协议	需要读取外部数据	查询数据库、发送 Slack 消息
Hooks	在特定事件触发时运行的确定性脚本（不涉及 LLM）	需要某件事每次自动发生	每次文件编辑后运行 ESLint
Plugins	将上述功能打包分发的容器	跨仓库复用、分发给他人	将一套配置打包为可安装插件

逐步构建配置

不需要一开始就配置所有内容，按需添加：

触发情境	添加内容
Claude 同一规范或命令犯错两次	添加到 CLAUDE.md
经常输入同样的提示词开始任务	保存为可调用的 Skill
反复粘贴同一套操作流程	捕获为 Skill
经常把浏览器中的数据复制给 Claude	将该系统接入 MCP
某项子任务把对话弄得杂乱无章	通过 Subagent 处理
希望某件事每次都自动发生	编写 Hook
另一个仓库需要同样的配置	打包为 Plugin

上下文占用成本

功能	加载时机	加载内容	上下文占用
CLAUDE.md	会话开始	全部内容	每次请求都有
Skills	会话开始 + 使用时	开始加载描述，使用时加载全文	低（仅描述）
MCP	会话开始	工具名称，按需加载完整 Schema	低（未用前）
Subagents	生成时	独立上下文 + 指定 Skills	与主会话隔离
Hooks	触发时	无（外部运行）	零（若 Hook 有返回值则除外）

三、.claude 目录结构

目录说明

Claude Code 从项目目录的 `.claude/` 文件夹和用户主目录的 `~/.claude/` 读取配置。

- **项目文件**（`.claude/`）：提交到 git，供团队共享
- **全局文件**（`~/.claude/`）：个人配置，适用于所有项目

主要配置文件

文件	作用	范围
CLAUDE.md	每次会话自动加载的指令	项目级/全局
rules/*.md	主题范围的指令，可按文件路径触发	项目级/全局
settings.json	权限、Hooks、环境变量、模型默认值	项目级/全局
skills/	可复用技能	项目级/全局
agents/*.md	子代理定义，含独立提示词和工具	项目级/全局
.mcp.json	MCP 服务器配置	仅项目级
~/.claude.json	应用状态、OAuth、UI 开关、个人 MCP	仅全局

如何选择正确的文件

需求	编辑文件
给 Claude 提供项目上下文和规范	CLAUDE.md
允许或禁止特定工具调用	settings.json 的 permissions 或 hooks
在工具调用前后运行脚本	settings.json 的 hooks
添加可通过 /name 调用的能力	skills/<name>/SKILL.md
定义具有独立工具的子智能体	agents/*.md
连接外部工具（MCP）	.mcp.json

应用数据（自动写入）

Claude Code 运行时会在 `~/.claude/` 下自动写入：

- **自动清理**（默认保留 30 天）：对话记录、子智能体记录、工具输出、文件快照、调试日志等
- **永久保留**：提示词历史（`history.jsonl`）、Token 统计（`stats-cache.json`）

⚠️ **安全提示**：对话记录和历史以明文存储，OS 文件权限是唯一保护。若 Claude 读取了 `.env` 文件或命令输出了凭据，这些信息会被写入记录文件。

清理项目数据：

代码块

```
1  claude project purge ~/work/my-repo          # 清理指定项目数据
2  claude project purge ~/work/my-repo --dry-run # 预览，不实际删除
```

四、上下文窗口详解

上下文窗口包含的内容

Claude Code 的上下文窗口在一次会话中按时间序列加载：

会话开始前（输入任何内容之前自动加载）：

- CLAUDE.md 内容
- 自动记忆（Auto Memory）
- MCP 工具名称
- Skills 描述

Claude 工作过程中：

- 每次文件读取都会增加上下文
- 路径范围的规则（path-scoped rules）在访问匹配文件时自动加载
- Hook（如 PostToolUse）在每次编辑后触发

上下文管理建议

Claude Code 接近上下文上限时会自动压缩：先清除旧的工具输出，必要时对会话做结构化摘要。你的请求和关键代码片段会保留；会话早期的详细指令可能丢失——所以持久规则要放在 CLAUDE.md，而不是依赖对话历史。

- 运行 `/context` 查看各类内容的上下文占用情况及优化建议
- 运行 `/memory` 检查启动时加载了哪些 CLAUDE.md 和自动记忆文件
- **CLAUDE.md 建议控制在 200 行以内**，参考资料移入 Skills（按需加载）

- 对于不希望 Claude 自动触发的 Skill，设置 `disable-model-invocation: true`，降低上下文占用至零
- 使用 Subagent 处理会产生大量中间输出的任务，只有摘要返回主上下文，保持主会话整洁

五、常用操作

会话管理

代码块

```
1  # 启动会话
2  claude                                # 在当前目录启动交互式会话
3  claude "帮我修复登录的 bug"          # 一次性任务，执行完退出
4  claude -c                             # 续上最近一次会话
5  claude -r                             # 打开列表，手动选择恢复哪个会话
6  claude -r <session-id>                # 直接恢复指定会话
7  claude -n "feature-auth"             # 启动并给会话命名（便于管理）
8  claude --list                         # 列出所有历史会话
9  claude --fork-session                 # 配合 --resume 使用，创建分支会话而非在原会话上继续
10 claude -p "问题"                      # 非交互模式，执行后退出（适合CI/脚本集成）
11 git log --oneline -20 | claude -p "summarize these recent commits" # 管道传入内容处理
12
13
14 # 后台会话管理
15 claude agents    # 打开所有后台会话的统一列表（或按左方向键）
16 /bg              # 将当前会话推到后台继续运行
17 claude --bg "任务描述" # 作为后台 agent 启动，立即返回并打印会话 ID
18 claude logs <id>    # 查看后台会话的最新输出
19 claude stop <id>    # 停止后台会话
20 claude rm <id>      # 从列表中移除后台会话
```

上下文管理（Slash命令）

命令	作用
/clear	清空全部对话历史，开始新任务用
/compact	将历史压缩成摘要，释放 token 但保留脉络
/context	查看当前上下文用量，接近上限时用
/rewind	回退到某条消息
/branch <名称>	创建对话分支

权限模式切换

- 1. 在 session 内按 `Shift+Tab` 循环切换（default → acceptEdits → plan）；
- 2. 启动时用 `--permission-mode <mode>` 指定；

代码块

```
1  claude --permission-mode default           # 每步确认（默认）
2  claude --permission-mode acceptEdits      # 文件改动自动通过
3  claude --permission-mode plan             # 先审计划再执行
4  claude --permission-mode bypassPermissions # 全自动（仅限容器、虚拟机环境）
```

- 3. 在 settings.json 中设置 `permissions.defaultMode` 作为永久默认值。

定时任务

可通过 Routines（云端定时）、Desktop scheduled tasks（本机定时）、GitHub Actions（CI 触发）、`/loop`（会话内轮询）四种方式实现定期自动任务。

常见问题

CLAUDE.md 和 MEMORY.md 的区别

每次 Claude Code 会话都从空白上下文开始。两种机制负责跨会话传递知识：

	CLAUDE.md	MEMORY.md
谁来写	你	Claude 自动写
内容	指令和规则	学到的规律和偏好
范围	项目、用户或组织级别	每个仓库（共享跨会话）
加载	每次会话完整加载	每次会话加载前 200 行或 25KB
适合存放	编码规范、工作流、项目架构	构建命令、调试发现、Claude 自主学到的偏好